LAB 1 (PART C)

COURSE CODE : **SAIA 3113**

COURSE NAME : **AI ENGINEERING**

YEAR/ PROGRAMME : **3 / BACHELOR OF ARTIFICIAL INTELLIGENCE**

SECTION : **1**

LECTURER'S NAME : **PROF. DR. SHAMSUL SAHIBUDDIN**

SEMESTER 1, SESSION 2026/2027

NAME (MATRIC NUMBER):

1 AMIRAH YASMINE FATHIAH BINTI MOHD REDZUAN (A24AI0097)

Step C1 — Review EduAdvisor and Understand GitHub as the Engineering Record (5 minutes)

Review the completed EduAdvisor example and use it as a benchmark for framing your own project.

ECGInsight is a proposed AI-powered educational assistant that helps users understand ECG heartbeat classification labels through natural-language explanations grounded in publicly available research documents.

The project will use GitHub as an engineering record to maintain version-controlled documentation, project requirements, data sources, prompts, evaluation results, and implementation progress.

Why GitHub?

GitHub supports traceability, reproducibility, documentation, and version control throughout the AI Engineering lifecycle. It enables changes in system design, prompts, data processing and evaluation to be recorded and reviewed.

Step C2 — Complete the Project Scoping Template (10 minutes)

Read the EduAdvisor example first. Then complete the template for your own project.

Week 1 project framing is preliminary. You are not expected to conduct a full literature review. The aim is to show that the problem is supported by evidence, identify a small number of relevant existing studies or solutions, and state clear initial objectives. These will be refined in Topic 2.

Template
Save as: docs/week1_project_scope.md

Project Name	ECGInsight: An AI-Powered Assistant for Explaining ECG Arrhythmia Classification Results
Domain	Artificial Intelligence / Healthcare Education / ECG Signal Interpretation / Retrieval-Augmented Generation (RAG)
Problem Statement	Electrocardiogram (ECG) heartbeat classification models can identify different heartbeat categories, but their predicted labels and performance metrics may be difficult for non-specialist users to understand. Although existing studies focus on improving ECG classification performance, translating classification results into accessible, evidence-grounded explanations remains a separate engineering challenge. ECGInsight proposes an AI-powered educational assistant that explains predefined ECG heartbeat classification labels using publicly

	available research and reference documents. By integrating Retrieval-Augmented Generation (RAG), source citations and safety guardrails, the system aims to provide understandable explanations while reducing unsupported information and preventing diagnostic or treatment-related recommendations.
Evidence of the Problem	Evidence 1 — Complexity of ECG heartbeat classification De Chazal, O'Dwyer and Reilly (2004) developed an automatic heartbeat classification approach using ECG morphology and heartbeat interval features. Their work used multiple heartbeat categories and quantitative classification measures, illustrating the technical nature of ECG classification results. PubMed Source: Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features Evidence 2 — Need for reliable ECG reference information The MIT-BIH Arrhythmia Database provides 48 approximately half-hour ECG recordings with expert-reviewed reference annotations. Its documentation supports classification research but is primarily technical rather than designed as a conversational explanation tool for learners. physionet.org Source: MIT-BIH Arrhythmia Database — PhysioNet These sources establish the technical nature of ECG classification. The specific need for a conversational educational assistant is the proposed engineering problem, rather than a user-demand finding directly proven by those studies.
Initial Related Work (at least 3)	**Study / Solution 1:** De Chazal et al. (2004) — Automatic Classification of Heartbeats Using ECG Morphology and Heartbeat Interval Features. **Source / Citation:** De Chazal, P., O'Dwyer, M., & Reilly, R. B. (2004). Automatic classification of heartbeats using ECG morphology and heartbeat interval features. *IEEE Transactions on Biomedical Engineering, 51*(7), 1196–1206. https://doi.org/10.1109/TBME.2004.827359

	Key Limitation: Focuses on ECG classification accuracy and performance rather than providing understandable explanations of classification results for non-expert users. **Study / Solution 2:** Moody & Mark (2005) — MIT-BIH Arrhythmia Database. **Source / Citation:** Moody, G. B., & Mark, R. G. (2005). MIT-BIH Arrhythmia Database. *PhysioNet*. https://doi.org/10.13026/C2F305 **Key Limitation:** Provides ECG recordings and expert annotations but does not offer an interactive AI assistant to explain heartbeat classification results. **Study / Solution 3:** Lewis et al. (2020) — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. **Source / Citation:** Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://arxiv.org/abs/2005.11401 **Key Limitation:** Introduces a general RAG framework but does not specifically address ECG classification explanations, medical information reliability, or domain-specific safety controls.
Engineering Gap / Limitation	Existing ECG classification approaches primarily focus on predicting heartbeat categories and evaluating model performance. Public ECG resources provide reference data and technical explanations, but they are not necessarily integrated into an accessible, document-grounded conversational assistant. ECGInsight addresses this gap by developing an educational AI system that retrieves relevant ECG reference information and generates understandable explanations of predefined classification labels, supported by citations and safety controls.
Initial Project Objectives (2–3)	O1: To design a document-grounded AI assistant for explaining predefined ECG heartbeat classification results using publicly available reference documents. O2: To develop a Retrieval-Augmented Generation (RAG) workflow that generates understandable ECG classification explanations with source citations and appropriate safety guardrails. O3: To evaluate the developed system in terms of factual correctness, groundedness, citation accuracy, response latency and safety.

Target Users	Students, academic researchers and beginner learners studying ECG signal analysis, heartbeat classification and machine learning applications in biomedical research. The system is intended for educational interpretation of research outputs, not for patients seeking diagnosis or treatment guidance.
Data Sources (at least 2)	The project will use the following publicly accessible resources. Source 1 — MIT-BIH Arrhythmia Database Technical documentation, heartbeat annotation definitions and reference data. PhysioNet Source 2 — American Heart Association: What Is an Arrhythmia? Public educational information about abnormal heart rhythms. American Heart Association American Heart Association Source 3 — De Chazal et al. (2004) Research on ECG heartbeat classification, features and classification categories. IEEE Research Article The project may also consult MedlinePlus — Electrocardiogram for general ECG educational context. MedlinePlus Only accessible materials permitted for reuse will be incorporated into the retrieval knowledge base. These links identify potential sources; they do not imply that all full-text contents have unrestricted redistribution rights
Similar to EduAdvisor	Both EduAdvisor and ECGInsight use Retrieval-Augmented Generation to provide conversational explanations grounded in approved reference documents. Both systems require document retrieval, prompt engineering, source citations, evaluation methods and responsible AI controls.

Different from EduAdvisor	EduAdvisor focuses on academic advising, including course prerequisites, registration and graduation requirements. ECGInsight focuses on explaining predefined ECG heartbeat classification labels and related research concepts. It uses biomedical research literature and ECG reference documentation rather than university policy documents, and requires stricter safeguards against medical diagnosis and treatment advice.
Responsible AI Risks (at least 2)	Risk 1 — Hallucination: The system may generate incorrect explanations of heartbeat classification results, potentially misleading users. To minimise this risk, the system will retrieve information from approved reference documents and provide supporting citations. Risk 2 — Medical Misinterpretation: Users may misunderstand educational explanations as clinical diagnoses. To prevent this, the system will clearly state its non-diagnostic purpose, avoid providing patient-specific diagnoses or treatment recommendations, and refer users to healthcare professionals when necessary. Risk 3 — Bias / Incomplete Context: The reference materials may not represent all ECG conditions or patient populations, potentially resulting in incomplete or biased explanations. The system will limit its responses to supported information and clearly communicate dataset limitations. Risk 4 — Privacy Leakage: Users may unintentionally share identifiable medical information when interacting with the system. To protect user privacy, the system will use publicly available research examples, avoid collecting patient-identifiable data, and minimise unnecessary data collection.

Step C3 — Set up your GitHub Repository (15 minutes)

⚠️ **GITHUB SAFETY**

Never commit API keys or tokens, passwords or credentials, personal or confidential data, copyrighted documents that cannot be redistributed, or restricted university information.

Step-by-step GitHub setup:

1. Go to github.com — sign in or create a free account
2. Click '+' (top right) → 'New repository'
3. Name: ai-engineering-[your-project-name] (e.g. ai-engineering-medifaq)
4. Set to Public (**Use Public only when the project data and artefacts are suitable for public sharing)**
5. — tick 'Add a README file' → click 'Create repository'
6. Click the green 'Code' button → copy the HTTPS URL

7. Open a terminal (Mac: Terminal app / Windows: Git Bash) → type:

```
git clone [paste your repository URL here]
cd ai-engineering-[your-project-name]
mkdir docs data src notebooks evaluation prompts

# Mac/Linux
touch docs/.gitkeep data/.gitkeep

# Windows (Git Bash)
touch docs/.gitkeep data/.gitkeep

git add .
git commit -m 'Week 1: Initial repository structure'
git push
```

Fill in your README.md — EduAdvisor completed example:

🎓 **EduAdvisor Example**

EduAdvisor README.md

```
# EduAdvisor — AI-Powered Academic Advising Assistant

> Answers student questions about courses, prerequisites, and graduation

> requirements, grounded in official university PDF documents.

## Problem Statement
Students struggle to get timely, accurate academic advice. Advisors are overloaded.
EduAdvisor provides instant, document-grounded answers 24/7.

## Evidence of the Problem
- Course, policy, and graduation information is distributed across multiple official documents.
- A single advising question may require students to consult more than one source.

## Initial Related Work / Existing Approaches
- Human academic advising — contextual, but availability depends on staff time.
- Static FAQ/policy pages — authoritative, but fragmented and difficult to navigate.
- General-purpose AI assistants — flexible, but may not be grounded in official university documents.

## Engineering Gap / Limitation
[State the practical or technical limitation your project addresses.]

## Project Objectives
- O1: Design a document-grounded AI academic advising solution.
- O2: Develop the solution with retrieval and appropriate guardrails.
- O3: Evaluate answer quality, groundedness, latency, and safety.

## Target Users
Undergraduate students planning course registration and graduation.

## Data Sources
- Course Catalogue PDF (university website)
- Student Handbook PDF (university website)
- Graduation Requirements PDF (faculty office)
```

```
## Weekly Progress
| Week | Focus | Deliverable | Status |
|------|-------|-------------|--------|
| W1 | System map + project scope | README + docs | ✅ |
```

Student Template
Save as: README.md

```
# [Your Project Name]

> [One sentence: what does your system do and for whom?]

## Problem Statement
[2–3 sentences: problem, who has it, why AI engineering is the right solution]

## Evidence of the Problem
- [Evidence 1 + source/citation]
- [Evidence 2 + source/citation]

## Initial Related Work / Existing Approaches
- [Study/solution 1 + source/citation + key limitation]
- [Study/solution 2 + source/citation + key limitation]
- [Study/solution 3 + source/citation + key limitation]

## Project Objectives
- O1: To ...
- O2: To ...
- O3: To ... (if applicable)

## Target Users
[Who will use this system?]

## Data Sources
- [Source 1: document name — URL]
- [Source 2: document name — URL]

## Weekly Progress
| Week | Focus | Deliverable | Status |
|------|-------|-------------|--------|
| W1 | System map + project scope | README + docs | ✅ |
```

Final commits for Week 1:

```
# Copy your files into docs/ first, then:
git add docs/week1_observation_sheet.md
git add docs/week1_system_map.png
git add docs/week1_project_scope.md
git add README.md
git commit -m 'Week 1: Observation sheet, system map, project scope, README'
```

```
git push
```

☐ Verify your repository on github.com — you should see /docs/ with three files and a completed README.md

☐ Submit your GitHub repository URL on the course portal

📦 TOPIC 1 LAB DELIVERABLE SUMMARY

Component	Filename	Location	Feeds into
Observation Report	week1_observation_sheet.md	/docs/	Project Proposal
Conceptual AI System Map	week1_system_map.png	/docs/	Proposal Presentation
Project Scope	week1_project_scope.md	/docs/	Topic 2 Requirements, Architecture and Evaluation Criteria
README	README.md	/ (root)	All subsequent weeks — updated weekly
GitHub URL	—	Course portal submission	Assessment record

Preparation for Week 2

- Finalise your domain, problem statement, and data sources — you will use them to refine the project framing and write requirements in Week 2.
- Refine your problem evidence and bring at least 1–2 credible sources that show the problem or need exists.
- Complete an initial scan of at least 3 relevant studies or existing AI solutions and note one limitation for each.
- Review your 2–3 initial project objectives. Each objective should be clear enough to connect later to a design, implementation, or evaluation activity.
- Identify at least 2 publicly accessible documents for your domain and record their source URLs.
- Download copies for course/local use only where permitted. Do not commit copyrighted, confidential, restricted, or personally identifiable data to a public repository.
- Set up your Google account for Colab (used from Week 3): accounts.google.com
- Think about: How will you know if your AI system is working? What does a 'good answer' look like for your use case? This becomes the starting point for evaluation criteria in Topic 2.
- Refine your Engineering Gap / Limitation so that it is clearly linked to the evidence and existing approaches identified in Week 1.